EPHEC
ÉCOLE SUPÉRIEURE
ENSEIGNEMENT POUR ADULTES

Année académique **2026-2027**

Support de cours - Programmation orientée objet - Java

Module TMP

Dates et heures

Fabriquer, comparer, décaler, mesurer et afficher des dates

EPHEC — Bachelier en informatique, niveau 2

Manipuler une date paraît simple tant qu'on ne s'y met pas. Combien de jours dans un mois ? Ça dépend. Combien de jours dans une année ? Ça dépend aussi. Le 31 février existe-t-il ? Non, mais rien n'empêche de l'écrire. Les programmes des années 90 stockaient souvent une date dans trois `int` et faisaient l'arithmétique eux-mêmes : c'est la source d'une quantité impressionnante de bogues, entre les mois de longueur variable, les années bissextiles et les passages d'année.

Le JDK fait ce travail pour vous, et le fait correctement, dans le package `java.time`. Votre part consiste à savoir quelle classe demander, et à connaître les quelques pièges que ce module passe en revue.

> **À la fin de ce module, vous saurez**
>
> - choisir entre `LocalDate`, `LocalTime` et `LocalDateTime`, et les fabriquer, les lire et les comparer ;
> - mesurer un écart, et choisir entre `ChronoUnit`, `Period` et `Duration` ;
> - expliquer pourquoi une date ne change jamais, et ce que cela implique ;
> - afficher des nombres et des dates proprement, d'une façon qu'un test peut vérifier ;
> - passer d'une date à son texte et inversement, sans laisser passer une date impossible.

TMP §1 — Les classes de `java.time`

Les trois classes du quotidien

Trois classes couvrent l'essentiel des besoins :

Classe	Ce qu'elle représente	Exemple
`LocalDate`	une date, sans heure	`2027-01-19`
`LocalTime`	une heure, sans date	`18:00`
`LocalDateTime`	les deux	`2027-01-19T18:00`

Le mot « `Local` » signifie qu'elles ignorent les fuseaux horaires : elles décrivent une date ou une heure telle qu'on la lit sur un calendrier ou une horloge murale, sans dire où. Pour un anniversaire ou un horaire de cours, c'est exactement ce qu'il faut. Pour un vol Bruxelles-Tokyo, il faudrait `ZonedDateTime`, qui dépasse ce cours.

Le fuseau de la machine n'intervient qu'une seule fois, dans `now()`, pour décider quelle date il est à cet instant. L'objet produit n'en garde aucune trace : un `LocalDateTime` déplacé d'un serveur à l'autre désigne toujours la même heure de calendrier.

Le package `java.time` fait partie de la bibliothèque standard : il n'y a rien à installer ni à déclarer dans le `pom.xml`. C'est ce qui le distingue des bibliothèques tierces, que Maven doit aller chercher (voir `TEC-PLA.1` du mémento technique).

Laquelle choisir ?

Le tableau suivant est la carte du module. Les deux dernières lignes font l'objet de `TMP §7` : retenez pour l'instant qu'il existe deux façons de mesurer un écart, et qu'elles ne se valent pas.

Ce que vous représentez	La classe
une date de naissance, une échéance	`LocalDate`
une heure d'ouverture	`LocalTime`

Ce que vous représentez	La classe
un rendez-vous, un horodatage	`LocalDateTime`
un écart en années, mois et jours	`Period`
un écart en heures, minutes et secondes	`Duration`

Fabriquer une date

La classe `LocalDate` n'a pas de constructeur public. On fabrique une date par une méthode statique, appelée sur la classe et non sur un objet : `now`, `of` ou `parse`. C'est le motif déjà rencontré au module `POO` avec les méthodes statiques des wrappers.

```
LocalDate aujourdhui = LocalDate.now();
LocalDate naissance  = LocalDate.of(1990, 5, 17);
LocalDate lue        = LocalDate.parse("1990-05-17");
```

La méthode `of` prend les nombres dans l'ordre naturel (année, mois, jour), et le mois va de 1 à 12, sans le décalage d'une case qu'on trouve dans de vieilles API. La méthode `parse` n'accepte que le format ISO, `AAAA-MM-JJ` ; tout autre format demandera un `DateTimeFormatter`, voir `TMP §8`.

Lire un morceau de date

Les accesseurs sont nombreux et leurs noms sont prévisibles : la complétion d'IntelliJ suffit à les trouver.

```
int annee          = naissance.getYear();
int jourDuMois     = naissance.getDayOfMonth();
DayOfWeek jour     = naissance.getDayOfWeek();
boolean bissextile = naissance.isLeapYear();
int joursDuMois    = naissance.lengthOfMonth();
```

La méthode `getDayOfWeek` ne retourne pas un nombre mais un `DayOfWeek`, qui est une énumération : un type dont les valeurs sont connues d'avance. Vous les étudierez au module `ENU` ; pour l'instant, il suffit de savoir qu'un `DayOfWeek` peut s'afficher et se comparer. Et `lengthOfMonth` répond à une question que vous ne voudrez jamais calculer vous-même.

Comparer deux dates

Les opérateurs `<` et `>` ne fonctionnent que sur les types primitifs. Une date est un objet : on la compare donc par des méthodes, comme on compare des chaînes avec `equals` plutôt qu'avec `==`.

```
naissance.isBefore(aujourdhui);   // true
naissance.isAfter(aujourdhui);    // false
naissance.isEqual(aujourdhui);    // false
```

Quelques exemples avec `LocalDate`

Objectif	Code
la date du jour	`LocalDate aujourdhui = LocalDate.now();`
une date donnée (15/06/2008)	`LocalDate.parse("2008-06-15");` ou `LocalDate.of(2008, 6, 15);`
la date de demain	`LocalDate demain = LocalDate.now().plusDays(1);`
aujourd'hui moins un mois	`LocalDate.now().minus(1, ChronoUnit.MONTHS);`
le jour de la semaine	`DayOfWeek jeudi = LocalDate.parse("2010-07-15").getDayOfWeek();`
année bissextile ?	`boolean bissextile = LocalDate.now().isLeapYear();`

Objectif	Code
nombre de jours du mois, de l'année	`aujourdhui.lengthOfMonth();` et `aujourdhui.lengthOfYear();`
nombre de jours entre deux dates	`long jours = ChronoUnit.DAYS.between(naissance, aujourdhui);`

Dans ce tableau, `now`, `parse` et `of` s'appellent sur la classe elle-même ; `MONTHS` et `DAYS` sont des constantes de `ChronoUnit`, qui désigne une unité de mesure et se trouve dans le package `java.time.temporal`. Toutes les autres méthodes s'appellent sur une date. Aucune ne modifie la date d'origine : `plusDays` ou `minus` retournent une nouvelle date (voir `TMP §3`).

L'heure seule, puis les deux à la fois

La classe `LocalTime` suit exactement les mêmes conventions : pas de constructeur, des méthodes statiques, des accesseurs prévisibles, `isBefore` et `isAfter` pour comparer.

```
LocalTime debut = LocalTime.of(18, 0);
LocalTime fin   = LocalTime.parse("21:35");
int heure       = debut.getHour();
```

Quelques constantes désignent des heures remarquables : `LocalTime.MIN` et `LocalTime.MIDNIGHT` valent toutes deux minuit, `LocalTime.NOON` midi, et `LocalTime.MAX` la dernière nanoseconde de la journée.

La classe `LocalDateTime` combine les deux, et sait retourner chacune de ses moitiés :

```
LocalDateTime rdv = LocalDateTime.of(2027, 1, 19, 18, 0);

rdv.toLocalDate();   // 2027-01-19
rdv.toLocalTime();   // 18:00
```

Retenez cette dernière possibilité : passer d'une classe à l'autre est souvent la réponse à une méthode qui refuse vos arguments.

> **Démonstration** - `Demo_TMP` : explorer une classe du JDK sans quitter l'éditeur
>
> La démonstration ne présente aucune API nouvelle. Elle montre comment trouver seul, dans l'éditeur, ce qu'on ne connaît pas encore : la complétion comme table des matières, la bulle de documentation comme réseau de liens, et le piège d'un nom qui ressemble à ce qu'on cherche sans en être. C'est la méthode que les énoncés supposent acquise chaque fois qu'ils écrivent « cherchez dans la Javadoc ». Voir `IDE.11` et `IDE.14` du guide IntelliJ.

TMP §2 — Mesurer un écart

Combien de jours entre deux dates ? La question a l'air anodine ; elle demande de choisir une unité, et de savoir dans quel sens compter.

`ChronoUnit`, l'unité de mesure

```
import java.time.temporal.ChronoUnit;

long jours = ChronoUnit.DAYS.between(naissance, aujourdhui);
long mois  = ChronoUnit.MONTHS.between(naissance, aujourdhui);
```

La tournure surprend : c'est l'unité qui contient la méthode `between`, et les deux dates sont ses arguments. On écrit `ChronoUnit.DAYS.between(...)`, et non `ChronoUnit.between.DAYS`. Lisez-la de

gauche à droite : « en jours, l'écart entre ceci et cela ». `ChronoUnit` propose `DAYS`, `WEEKS`, `MONTHS`, `YEARS`, et une dizaine d'autres unités.

Deux pièges de comptage

- `between` **compte des unités entières.** Entre le 1er et le 30 janvier, il n'y a pas un mois entier : `MONTHS.between` retourne 0. Ce n'est pas un défaut, c'est une définition, et il faut la connaître avant de s'en étonner.
- **L'ordre des arguments décide du signe.** Inverser les deux dates inverse le signe du résultat, qui peut donc être négatif.

Le résultat est un `long`, pas un `int` : un écart exprimé en secondes déborde très vite la capacité d'un `int`. La question revient plus loin (voir `TMP §5`).

> **Exercices** - `TMP.1` Mesurer un écart entre deux dates
>
> Quatre méthodes statiques mesurent le même écart dans quatre unités. Elles ne lisent rien au clavier et n'affichent rien : elles reçoivent deux dates et retournent un nombre. C'est la séparation entre le calcul et le dialogue, posée au module `POO` et appliquée partout ensuite.

TMP §3 — Une date ne change pas

Décaler une date

Les méthodes qui déplacent une date existent par dizaines, et leurs noms ne posent aucune difficulté : `plusDays`, `minusMonths`, `withDayOfMonth`. Leur comportement, en revanche, mérite qu'on regarde de près. Avant de lire la suite, devinez ce qu'affiche la dernière ligne, puis exécutez le code pour vérifier.

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);

rendezVous.plusWeeks(1);
System.out.println(rendezVous);   // ?
```

> **Exercices** - `TMP.2` Les dates sont immuables
>
> Exécutez d'abord le programme fourni, et lisez sa sortie. Il contient une erreur qui ne provoque ni refus du compilateur, ni plantage, ni message d'aucune sorte, et donne pourtant un résultat faux.

Une date est immuable

La ligne affiche `2027-01-19` : la date n'a pas bougé. Comme un `String`, un `LocalDate` est **immuable** : sa valeur est fixée à la construction et ne changera jamais. La méthode `plusWeeks` ne modifie rien ; elle fabrique un nouvel objet et le retourne. Si ce nouvel objet n'est pas affecté à une variable, il est perdu.

```
LocalDate rendezVous = LocalDate.of(2027, 1, 19);
LocalDate uneSemainePlusTard = rendezVous.plusWeeks(1);   // 2027-01-26
```

Vous aviez déjà rencontré cette mécanique sans la nommer : `texte.toUpperCase()` ne met pas la chaîne en majuscules, elle en retourne une autre.

Ce n'est pas une gêne, c'est une garantie. Vous pouvez confier un objet immuable à n'importe quelle méthode sans craindre qu'elle le modifie dans votre dos : c'est ce que réclame `CC-MET.7` du mémento Clean Code, qui met en garde contre les effets de bord.

> **À retenir**
>
> Les méthodes d'une date retournent une nouvelle date ; elles ne modifient jamais celle sur laquelle on les appelle. Affectez toujours leur résultat.

TMP §4 — Un détour par l'affichage

Les exercices qui suivent produisent des nombres longs et des durées, et `println` ne suffira plus. Cette section est le seul moment de l'année consacré à l'affichage formaté ; tout ce qui vient après s'en sert, y compris le module `TAB`.

`println` et `print`

```
System.out.println("Bonjour");   // affiche et passe a la ligne
System.out.print("Bonjour");     // affiche, sans passer a la ligne

System.out.print("A");
System.out.print("B");           // => AB, sans passage a la ligne après
```

La méthode `print` fait la même chose que `println`, sans le retour à la ligne. Elle sert dès qu'on construit une ligne morceau par morceau : l'affichage d'une matrice, par exemple, au module `TAB`.

`printf` : un gabarit et des valeurs

La méthode `printf` prend un gabarit contenant des « trous à remplir », marqués par `%`, puis les valeurs qui viennent remplir ces trous, dans l'ordre : le premier trou reçoit la première valeur.

```
System.out.printf("%s a %d ans%n", "Max", 21);
// Max a 21 ans
```

Pour un affichage simple, ce n'est pas plus court qu'une concaténation ; dès qu'il y a trois valeurs ou une question de présentation, c'est beaucoup plus lisible, et souvent plus court. Employez `%n` plutôt que `\n` : `%n` produit le retour à la ligne de la machine sur laquelle le programme tourne.

Gabarit	Résultat	Effet
`%d`	`1234567`	un entier
`%,d`	`1,234,567`	séparateur de milliers
`%s`	`Max`	du texte
`%f`	`3.141590`	six décimales par défaut
`%.2f`	`3.14`	deux décimales
`%8.2f`	`    3.14`	deux décimales, sur 8 caractères
`%n`	(saut de ligne)	le retour à la ligne de la machine
`\t`	(tabulation)	pas un gabarit : un caractère de tabulation, utilisable dans toute chaîne

Le gabarit `%8.2f` sert à aligner une colonne de nombres : chaque valeur occupe la même largeur, et les séparateurs décimaux se superposent.

La locale décide du séparateur

Sans précision, le séparateur des milliers et celui des décimales dépendent de la langue de la machine :

```
System.out.printf("%,d%n", 1157479200);
// machine francaise : 1 157 479 200  (espace INSECABLE etroite)
// machine anglaise  : 1,157,479,200

System.out.printf(Locale.US, "%,d%n", 1157479200);
// partout           : 1,157,479,200
```

C'est le piège le plus coûteux de cette section, parce qu'il ne se voit pas. Le JDK français sépare les milliers par un espace insécable étroit, qui ressemble à s'y méprendre à un espace ordinaire. Un test qui compare le texte produit échouera sur une différence invisible à l'œil, et vous chercherez l'erreur ailleurs.

> **À retenir**
>
> Dès que le texte produit est comparé par un test, précisez la locale.

> **Pour aller plus loin** - une espace qui n'en est pas une
>
> Le séparateur de la locale française est l'espace fine insécable, `\u202F`. Pour Java, ce n'est pas un « blanc » : `Character.isWhitespace('\u202F')` retourne `false`, comme pour toutes les espaces insécables, qui par définition ne doivent pas couper un mot. Découper `"1 157 479 200"` produit par `String.format` sur ses espaces, avec `split(" ")` ou `isWhitespace`, ne donne donc qu'un seul morceau. Voir aussi `POO §7`, à propos de `Character.isWhitespace`.

`String.format`, la version testable

La méthode `String.format` prend exactement les mêmes gabarits que `printf`. La différence est décisive pour la suite du cours :

```
// affiche la valeur a la console
System.out.printf(Locale.US, "%,d", n);

// retourne la valeur a l'appelant
String s = String.format(Locale.US, "%,d", n);
```

- `printf` écrit dans la console et ne retourne rien : aucune classe de test ne peut vérifier son travail ;
- `String.format` retourne une chaîne, qu'un test peut comparer.

C'est la règle posée au module `POO`, reformulée pour l'affichage : le noyau retourne une valeur, le `main` se contente de dialoguer. Vos méthodes de formatage retourneront donc des `String`, et seul le `main` appellera `printf`.

TMP §5 — Date ou instant ?

Ce que `LocalDate` ne contient pas

Un `LocalDate` désigne une case du calendrier, le 17 mai 1990, et rien d'autre : pas d'heure, pas de minute, pas de seconde. Que donne alors la seconde ligne ?

```
ChronoUnit.DAYS.between(d1, d2);      // fonctionne
ChronoUnit.SECONDS.between(d1, d2);   // ?
```

> **Exercices** - `TMP.3` Compter des secondes : date ou instant ?
>
> L'exercice reprend `TMP.1` en changeant l'unité. Décommentez la ligne signalée et lisez le message : la capacité à lire un message d'erreur vaut mieux qu'une réponse apprise. Le formatage du résultat emploie ce qui vient d'être vu en `TMP §4`.

Le JDK refuse plutôt que d'inventer

La seconde ligne lève une exception : `Unsupported unit: Seconds`. Demander à deux cases de calendrier combien de secondes les séparent n'a pas de sens, et le JDK refuse plutôt que de choisir une heure à votre place.

La méthode `atStartOfDay()` fabrique le `LocalDateTime` qui correspond à minuit. C'est alors vous qui prenez la décision, et elle reste une convention, pas une vérité : votre résultat est juste à quelques heures près.

Le débordement annoncé en `TMP §2` se vérifie ici : un `int` s'arrête à 2 147 483 647, soit environ 68 ans de secondes. Au-delà, il faut un `long`.

TMP §6 — L'arithmétique du calendrier

Ajouter un mois n'est pas ajouter trente jours. Les deux dernières lignes donnent-elles la même date ? Décidez-le avant d'exécuter.

```
LocalDate d = LocalDate.of(2027, 1, 31);

d.plusDays(30);
d.plusMonths(1);
d.plusMonths(1).plusMonths(1);   // ?
d.plusMonths(2);                 // ?
```

> **Exercices** - `TMP.4` L'arithmétique des mois
>
> Deux méthodes ; la seconde compare l'ajout d'un coup et l'ajout un par un, et retourne un booléen. L'énoncé donne les quatre cas à observer, en année ordinaire et en année bissextile.

Java rabote au dernier jour du mois

Le 31 janvier 2027 plus un mois donne le 28 février 2027 : le 31 février n'existe pas, et Java rabote au dernier jour disponible du mois d'arrivée. Une fois raboté, le 31 est perdu. La seconde addition repart du 28, et non du 31 : ajouter deux fois un mois n'est donc pas ajouter deux mois.

Quelle autre réponse aurait été possible ? Le 3 mars ? Une exception ? Chacune a ses inconvénients.

> **À retenir**
>
> Pour une échéance, décidez si vous comptez en jours ou en mois, et dites-le.

TMP §7 — `Period` ou `Duration`

Deux classes mesurent un écart. Elles ne mesurent pas dans la même unité de pensée, et les confondre produit du code qui ne compile pas, ce qui, pour une fois, est une bonne nouvelle.

`Period`, les unités du calendrier

```
Period p = Period.between(naissance, aujourdhui);
p.getYears();    // 36
p.getMonths();   // 8
p.getDays();     // 2
```

La classe `Period` décompose l'écart en années, mois et jours, exactement comme on annonce un âge. Elle travaille sur des `LocalDate`. Ces unités n'ont pas de longueur fixe : un mois vaut 28, 29, 30 ou 31 jours. Un `Period` ne mémorise donc pas une quantité de temps, mais une recette d'ajustement du calendrier.

C'est la réponse promise au module `POO`. La méthode `getAge()` ne connaissait que l'année de naissance : une personne née en décembre était déclarée plus âgée qu'elle ne l'est, dès le mois de janvier. Avec la date de naissance complète, `Period.between(naissance, LocalDate.now()).getYears()` donne l'âge exact.

`Duration`, les unités de l'horloge

```
Duration d = Duration.between(debut, fin);
d.toHours();     // 3
d.toMinutes();   // 215
```

La classe `Duration` compte en heures, minutes et secondes, des unités qui ne varient jamais. Elle travaille sur des `LocalDateTime` (ou des `LocalTime`).

La règle tient en une ligne : `Period` pour des dates, `Duration` pour des heures. Le compilateur la fait respecter, puisque chacune refuse le type de l'autre.

> **Exercices** - `TMP.5 Period` ou `Duration`
>
> Trois méthodes. La troisième est celle qui compte : écrire naïvement `toHours() + "h" + toMinutes()` ne donne pas ce qu'on attend, et la Javadoc de `Duration` contient la méthode qui règle le problème. C'est exactement la démarche montrée en démonstration.

Le total et le reste

`toHours() + "h" + toMinutes()` donne `3h215` : `toMinutes()` retourne le total des minutes, 215. C'est `toMinutesPart()` qui retourne le reste après extraction des heures, 35. Toute la famille `toXxx` retourne un total ; toute la famille `toXxxPart` retourne un reste.

Au-delà de `Duration`, retenez la leçon de méthode : deux noms voisins dans la même liste de complétion peuvent faire des choses différentes, et rien dans la liste ne le dit. La complétion montre les noms ; la Javadoc dit ce qu'ils font.

TMP §8 — Une date et son texte

Une date affichée au format ISO se lit mal, et une date saisie par un utilisateur n'est jamais au format ISO. Cette section fait le passage dans les deux sens.

De la date vers le texte

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String texte = date.format(fr);    // "19/01/2027"
```

Le motif est une chaîne où chaque lettre désigne un champ, et le nombre de lettres la largeur souhaitée : `dd` le jour, `MM` le mois, `yyyy` l'année.

> **Attention**
>
> `mm` est la minute, `MM` le mois. La confusion ne provoque aucune erreur : elle donne simplement une date fausse.

Du texte vers la date

Le même formateur sert dans les deux sens : `date.format(fr)` transforme une date en texte, et `LocalDate.parse(texte, fr)` fait l'inverse, en lisant le texte selon le même motif. Cette lecture s'appelle l'analyse (*parsing* en anglais).

```
DateTimeFormatter fr = DateTimeFormatter.ofPattern("dd/MM/yyyy");
LocalDate d = LocalDate.parse("19/01/2027", fr);
LocalDate e = LocalDate.parse("19-01-2027", fr);   // exception
```

Un texte qui ne correspond pas au motif lève une `DateTimeParseException` : le programme s'arrête et affiche une trace. Traiter proprement ce genre d'incident est le sujet du module `EXC` ; pour l'instant, contentez-vous de constater ce qui se passe.

> **Exercices** - `TMP.6` Formater et analyser une date
>
> Trois méthodes : formater, analyser, et analyser en refusant les dates impossibles. Analysez `"31/02/2027"` et regardez ce que vous obtenez avant de lire la suite.

L'indulgence silencieuse

L'appel `parse("31/02/2027")` retourne `2027-02-28`, sans rien signaler. Par défaut, le formateur est indulgent : il rabote. Une saisie absurde vient de devenir une date parfaitement plausible, et plus rien dans le programme ne permettra de s'en apercevoir. C'est exactement ce que proscrit `CC-ERR.1` du mémento Clean Code : mieux vaut une exception qu'une valeur arrangée.

Le mode strict refuse les dates impossibles. Il s'obtient avec `withResolverStyle(ResolverStyle.STRICT)`, et réclame alors le motif `uuuu` au lieu de `yyyy` pour l'année. Le mode indulgent n'est pas un défaut du JDK : il existe des contextes où ajuster vaut mieux que refuser. Mais pour un formulaire d'inscription, le mode strict est le bon choix, et il faut l'écrire explicitement.

Ce qu'il faut retenir

- Trois classes : `LocalDate`, `LocalTime`, `LocalDateTime`.
- Une date est immuable : les méthodes retournent une nouvelle date, elles ne modifient pas.
- Une date n'est pas un instant : on ne compte pas de secondes entre deux cases du calendrier.
- `Period` pour des dates, `Duration` pour des heures.
- `String.format` se teste, `printf` non.
- Précisez la locale, et choisissez le mode strict.

La plus transversale de ces règles est la deuxième : l'immuabilité dépasse largement les dates. Vous la retrouverez sur `String`, sur les `record` du module `ENU`, et dans le mémento Clean Code.